# Document réponse - DS N°3

**Durée: 2 heures** **2025-2026** **21/03/2026**

## Optimisation d'un run de Ski Freestyle (JO d'Hiver)

L'utilisation de toute calculatrice, de tout matériel électronique et de tout document est interdite

# 1. Contexte

## 1.1. Modélisation des données

# 2. Échauffement et Manipulations / 4.5 points

**Q1.** **/0.5**

Sortie du code :

**Correction Q1.** Le code affiche : `24` (car `15 + 9 = 24`).

**Q2.** **/1**

Fonction `score_total(combo: list) -> int`

**Correction Q2.**

```
def score_total(combo):
    total = 0
    for figure in combo:
        total += figure[2]
    return total
```

**Q3.** **/1**

Fonction `est_valide(combo: list, temps_vol: float) -> bool`

**Correction Q3.**

```
def est_valide(combo, temps_vol):
    duree_totale = 0
    for figure in combo:
        duree_totale += figure[1]
    return duree_totale <= temps_vol
```

**Q4.** **/2**

Fonction `figures_possibles(repertoire: list, temps_vol: float) -> list`

**Correction Q4.**

```python
def figures_possibles(repertoire, temps_vol):
    resultat = []
    for figure in repertoire:
        if figure[1] <= temps_vol:
            resultat.append(figure)
    return resultat
```

La complexité est $\boldsymbol{O(n)}$ car on parcourt une seule au maximum fois la liste des $n$ figures.

## 3. Tableau d'affichage (Conversion binaire) / 3.5 points

**Q5.** **/0.5**

Combien de bits faut-il pour stocker le score d'un skieur en complément à 2 ?

**Correction Q5.** Un score de 100 nécessite 7 bits en binaire pur ($2^7 = 128$). En complément à 2, il faut un bit supplémentaire pour le signe. Avec **8 bits**, on peut coder de $-128$ à $+127$, ce qui est suffisant.

**Q6.** **/1.5**

Complément à 2 :

1. $44_{(10)}$
2. $-32_{(10)}$

**Correction Q6.** 8 bits sont utilisés :

1. $44 = 32 + 8 + 4 = 00101100_2$
2. $+32 = 00100000_2$. Inversion $(11011111) + 1 = \mathbf{11100000_2}$

**Q7.** **/1.5**

Décimales des complément à 2 :

- $1111\ 0101_{(2)}$
- $0011\ 0101_{(2)}$

**Correction Q7.**

- $11110101_2$ : Bit de poids fort à 1 (négatif). Inversion $(00001010) + 1 = 00001011 = 11$. Donc $\mathbf{-11}$.
- $00110101_2$ : Bit de poids fort à 0 (positif). $32 + 16 + 4 + 1 = \mathbf{53}$.

## 4. Analyse des passages (Dictionnaires) / 3.5 points

**Q8.** /1.5

Fonction `cumul_points(passages: list) -> dict`

**Correction Q8.**

```
1 def cumul_points(passages):
2     scores = {}
3     for nom, points in passages:
4         if nom in scores:
5             scores[nom] += points
6         else:
7             scores[nom] = points
8     return scores
```

**Q9.** /2

Fonction `vainqueur(dico_scores: dict) -> str`

**Correction Q9.**

```
1 def vainqueur(dico_scores):
2     v = None
3     for nom in dico_scores:
4         if v is None or dico_scores[nom] > dico_scores[v]:
5             v = nom
6     return v
```

## 5. Recherche dans le répertoire (Dichotomie) / 4 points

**Q10.** /2.5

Fonction `dicho_rec(repertoire: list, nom: str) -> bool`

**Correction Q10.**

```
1  def dicho_rec_aux(liste, nom, g, d):
2      if g > d:
3          return False
4      m = (g + d) // 2
5      if liste[m][0] == nom:
6          return True
7      elif liste[m][0] < nom:
8          return dicho_rec_aux(liste, nom, m + 1, d)
9      else:
10         return dicho_rec_aux(liste, nom, g, m - 1)
11
12 def dicho_rec(repertoire, nom):
13     return dicho_rec_aux(repertoire, nom, 0, len(repertoire) - 1)
```

**Q11.** /0.5

Nombre d'appels récursifs au maximum pour un répertoire de 1024 figures :

**Correction Q11.** $1024 = 2^{\{10\}}$. À chaque étape, on divise la taille par 2. Il faudra donc au maximum **10 appels** (profondeur de l'arbre binaire).

**Q12.** /1

Complexité de la fonction `dicho_rec` :

**Correction Q12.** On pose $n$ la taille du répertoire. À chaque appel récursif, on divise l'espace de recherche par 2. On a donc la relation de récurrence $C(n) = C(\frac{n}{2}) + O(1)$. D'après le Master Theorem, cela donne une complexité de $O(\log n)$.

# 6. Stratégies d'optimisation

## 6.1. La stratégie du « Spectaculaire » (Glouton par points) / 6.5 points

**Q13.** /2

Fonction `trier_par_points(repertoire: list) -> list`

**Correction Q13.**

```python
def trier_par_points(repertoire):
    rep_tri = list(repertoire) # Copie
    for i in range(len(rep_tri)):
        max_idx = i
        for j in range(i + 1, len(rep_tri)):
            if rep_tri[j][2] > rep_tri[max_idx][2]:
                max_idx = j
        rep_tri[i], rep_tri[max_idx] = rep_tri[max_idx], rep_tri[i]
    return rep_tri
```

**Q14.** /1

Nom de l'algorithme de tri utilisé et sa complexité :

**Correction Q14.** L'algorithme utilisé est le **tri par sélection**. Sa complexité est $\boldsymbol{O(n^2)}$ car on utilise deux boucles imbriquées parcourant chacune environ $n$ éléments.

**Q15.** /2.5

Fonction `glouton_points(repertoire: list, temps_vol: float) -> list`

**Correction Q15.**

```python
def glouton_points(repertoire, temps_vol):
    rep_tri = trier_par_points(repertoire)
    combo = []
    temps_restant = temps_vol
    i = 0
    while i < len(rep_tri):
        if rep_tri[i][1] <= temps_restant:
            combo.append(rep_tri[i])
            temps_restant -= rep_tri[i][1]
        else :
            i += 1
    return combo
```

**Q16.** /1

Est-ce que cette stratégie est optimale ? Si non, donner un contre-exemple.

**Correction Q16.** **Non**, elle n'est pas optimale.
**Contre-exemple :**
Temps de vol = 4s.
Figures : A (3s, 10pts), B (2s, 6pts). L'algorithme glouton prend A (10pts). L'optimal prend B+B (12pts).

## 6.2. La stratégie de « L'Efficace » (Glouton par ratio) / 2.5 points

**Q17.** /0.5

Fonction `ratio(figure: n-uplet) -> float`

**Correction Q17.**

```python
def ratio(figure):
    return figure[2] / figure[1]
```

**Q18.** /0.5

Changement à effectuer sur la fonction `glouton_points` pour obtenir la fonction `glouton_ratio`.

**Correction Q18.** Il suffit de remplacer l'appel à `trier_par_points(repertoire)` par un appel à `trier_par_ratio(repertoire)`.

**Q19.** **/1.5**

Contre-exemple explicite où l'algorithme par ratio n'atteint pas le score maximal possible.

**Correction Q19.** **Contre-exemple :** Temps de vol = 10s.

- Figure A : Durée 6s, Points 60 (Ratio 10)
- Figure B : Durée 5s, Points 45 (Ratio 9)

L'algorithme par ratio prend A (60 pts). L'optimal prend B + B (90 pts).

## 6.3. La Solution Parfaite (Programmation dynamique) / 4.5 points

**Q20.** **/1.5**

Représentation de l'arbre $f(i, t)$ et score final :

**Correction Q20.**

```
f(0, 4.0)
├── f(1, 4.0)
│   ├── f(2, 4.0)
│   │   ├── f(3, 4.0) -> 0
│   │   └── 10 + f(3, 2.0) -> 10
│   └── 10 + f(2, 2.0)
│       ├── f(3, 2.0) -> 0
│       └── 10 + f(3, 0.0) -> 10
│       └── Total : 20
└── 12 + f(1, 1.5)
    ├── f(2, 1.5)
    │   └── f(3, 1.5) -> 0
    └── Total : 12
```

**Score final : 20**

**Q21.** **/2**

Fonction récursive

```
meilleur_score_rec(rep: list, t : float, i: int) -> int
```

**Correction Q21.**

```python
def meilleur_score_rec(rep, t, i):
    if i >= len(rep):
        return 0
    nom, duree, points = rep[i]
    if duree > t:
        return meilleur_score_rec(rep, t, i + 1)
    else:
        # On choisit le max entre ne pas prendre la figure ou la prendre
        option1 = meilleur_score_rec(rep, t, i + 1)
        option2 = points + meilleur_score_rec(rep, t - duree, i + 1)
        return max(option1, option2)
```

**Q22.** **/1**

Complexité temporelle dans le pire cas :

**Correction Q22.**

On pose $n$ la taille de la liste rep. À chaque appel, on effectue potentiellement deux nouveaux appels récursifs. Et la complexité de traitement est $O(1)$. On a donc la relation de récurrence : $C(n) = 2C(n-1) + O(1)$

$C(n) = O(2^n)$.

## 7. Bonus / 2 points

**Q23.** **/2**

Fonction de tri en $O(n \log n)$ :

**Correction Q23.**

```python
def fusion(gauche, droite):
    resultat = []
    i = j = 0
    while i < len(gauche) and j < len(droite):
        if gauche[i][2] > droite[j][2]:
            resultat.append(gauche[i])
            i += 1
        else:
            resultat.append(droite[j])
            j += 1
    resultat += gauche[i:]
    resultat += droite[j:]
    return resultat

def tri_fusion(rep):
    if len(rep) <= 1:
        return rep
    milieu = len(rep) // 2
    gauche = tri_fusion(rep[:milieu])
    droite = tri_fusion(rep[milieu:])
    return fusion(gauche, droite)
```